Diagramme de classes UML

Définition du diagramme de classes

- Le **diagramme de classes** est un **diagramme structurel UML** qui décrit la **structure statique** d'un système.
- Il montre :
  - **Les classes** du système.
  - **Leurs attributs et méthodes**.
  - **Les relations entre classes** (association, héritage, composition, etc.).

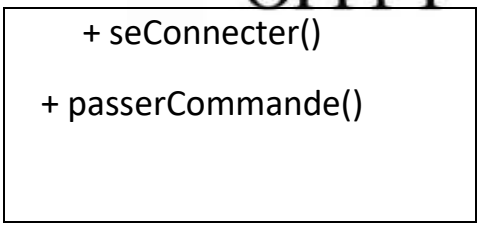
Exemple concret : dans une **application e-commerce**, on peut avoir les classes Client, Produit, Commande.

Visibilité et encapsulation

- **Encapsulation** : principe de cacher les détails internes et de contrôler l'accès aux données.
- UML utilise des symboles pour indiquer la visibilité :
  - **+ public** → accessible partout.
  - **- private** → accessible uniquement dans la classe.
  - **# protected** → accessible dans la classe et ses sous-classes.

Classe Client
- nom : String - motDePasse : String

Formateur : AARAB HAFID



Attributs et méthodes d'instance

- **Attributs d'instance** : chaque objet a sa propre valeur.
  - Exemple : chaque Client a un nom et une adresse.
- **Méthodes d'instance** : dépendent de l'objet.
  - Exemple : payer() agit sur un seul client.

Ex : Chaque objet CompteBancaire a son propre solde et ses propres opérations.

Classe CompteBancaire	Attributs	Méthodes
	- solde : Double	+ deposer() + retirer()

Attributs et méthodes de classe (static)

- Partagés entre **toutes les instances** de la classe.
- Notés en UML par le **soulignement**.
- Exemple :
  - Attribut tauxTVA
  - Méthode getNombreProduits() .

Classe Produit	Attributs	Méthodes
	- nom : String - prix : Double *tauxTVA : Double*	+ calculerPrixTTC() *+ getNombreProduits()*

Contraintes

- Règles spéciales à respecter.
- Notées entre {} en UML.
- Exemple :
  - {solde >= 0} dans la classe CompteBancaire.
  - {dateFin >= dateDébut} dans Contrat.

Attributs dérivés

- Attribut calculé automatiquement à partir d'autres attributs.
- Noté avec un /.
- Exemple :
  - /age dérivé de dateNaissance.
  - /montantTotal dérivé de la somme des produits d'une Commande.

Classe Personne	Attributs	Attribut dérivé
	- nom : String - dateNaissance : Date	/age (calculé à partir de dateNaissance)

Classe Commande	Attributs	Attribut dérivé
	- listeProduits : Produit[]	/montantTotal (somme des prix des produits)

Multiplicité (cardinalité)

- Indique combien d'objets peuvent participer à une relation.
- Notations UML :
  - 1 : un seul.
  - 0..1 : optionnel.
  - * : plusieurs.
  - 1..* : au moins un.

◇ **Exemple :**

- Un **Client (1)** peut passer **plusieurs Commandes (0..*)**.
- Une **Commande (1)** concerne **un seul Client (1)**.

Constructeurs et destructeurs

- **Constructeur** : initialise un objet.
- **Destructeur** : détruit un objet.
- En UML, on peut les mentionner comme méthodes spéciales.

◇ **Exemple :**

+ Client()

+ ~Client()

Modèles de classe

- Un **modèle de classe** = une représentation standardisée d'une classe.

- Contient :
  1. Nom de la classe.
  2. Attributs.
  3. Méthodes.

◇ **Exemple UML :**

Classe Client	Attributs	Méthodes
	- nom : String - email : String	+ seConnecter() + passerCommande()

Relations entre classes

a) Dépendance

- Une classe dépend d'une autre si elle l'utilise temporairement.
- Exemple : Facture dépend de Paiement.

b) Association

- Lien structurel permanent entre classes.
- Exemple : Client — Commande.

c) Association n-aire

- Relation entre plus de 2 classes.
- Exemple : Élève — Cours — Professeur.

d) Classe association

- Ajout d'une classe intermédiaire pour stocker des infos sur la relation.
- Exemple : Inscription entre Étudiant et Cours.

e) Composition

- Relation **forte** : la vie d'un objet dépend de l'autre.
- Exemple : Voiture — Moteur.

f) Agrégation

- Relation **faible** : une classe peut exister indépendamment.
- Exemple : Bibliothèque — Livre.

g) Héritage

- Une classe hérite d'une autre (relation généralisation/spécialisation).
- Exemple : Employé → Professeur et Administratif.

Classes concrètes et abstraites

- **Classe concrète** : peut être instanciée.
- **Classe abstraite** : notée en *italique*, ne peut pas être instanciée. Sert de modèle.

◇ **Exemple :**

- Classe abstraite Animal.
- Classes concrètes : Chien, Chat.

Relation de réalisation et interfaces

- Une **interface** définit un ensemble de méthodes sans implémentation.
- Une **classe concrète** réalise l'interface.

- **Exemple :**
    - Interface Payable → méthode payer().
    - Classe Commande réalise Payable.